Semáforo rudimentar sobre sockets TCP

Controle de acesso concorrente a um recurso distribuído

Disciplina	Sistemas Distribuídos — Comunicação e Concorrência
Trabalho	Implementação de um semáforo rudimentar para controlar o acesso a recursos distribuídos de forma concorrente
Aluno	Kayruan da Cruz Soares
Linguagem	C (sockets TCP, threads e semáforos)
Plataformas	Windows (Winsock2) e Linux (BSD sockets + pthreads)

1. Objetivo

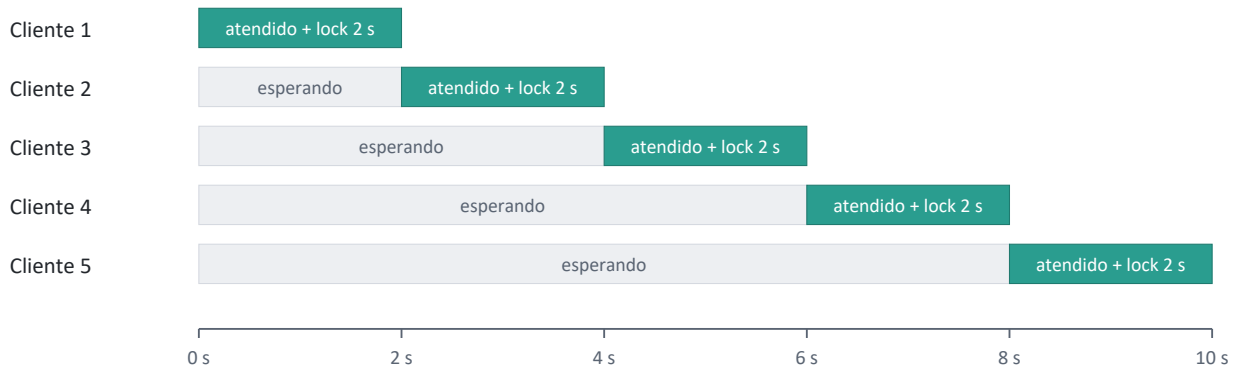
Implementar um semáforo rudimentar para controlar o acesso a um recurso distribuído de forma concorrente. Em termos práticos: **N clientes** se conectam simultaneamente a um servidor e solicitam a hora local dele. O servidor atende um cliente por vez e, após cada atendimento, mantém o recurso bloqueado por **2 segundos** — de modo que o 2º cliente só seja atendido dois segundos após o 1º, o 3º dois segundos após o 2º, e assim sucessivamente.

O ponto central é que os clientes chegam *ao mesmo tempo*, mas são servidos *em fila*. Quem impõe essa ordem é o semáforo.

1.1 O que é o "recurso"

O recurso protegido é o relógio do servidor — a leitura da hora local que será devolvida ao cliente. Não é um recurso caro de verdade; ele cumpre, no trabalho, o papel de qualquer seção crítica que não possa ser acessada por dois processos ao mesmo tempo.

1.2 Comportamento esperado



Cinco clientes disparados simultaneamente. As faixas cinzas são a espera no semáforo; as faixas verdes, o atendimento somado aos 2 s em que o recurso segue bloqueado.

2. Estrutura do projeto

```
Trabalho CP/
|-- src/
|   |-- servidor.c          servidor multithread com semaforo
|   |-- cliente.c           cliente de tempo
|-- scripts/
|   |-- compilar.ps1        compila src/ -> bin/ (Windows)
|   |-- rodar_demo.ps1      sobe o servidor e dispara N clientes
|   |-- gerar_pdf.py        gera esta documentacao
|-- docs/
|   |-- Documentacao_Trabalho.pdf   esta documentacao
|   |-- Trabalho.md                 enunciado do trabalho
|-- bin/                    binarios gerados pela compilacao
|-- Makefile                build e demo no Linux/WSL
|-- README.md               documentacao do repositorio
```

Apenas dois arquivos-fonte. O servidor tem cerca de 330 linhas e o cliente, 200 — ambos comentados. Cada um compila de forma independente nas duas plataformas, sem biblioteca externa alguma.

3. Como compilar e executar

O projeto compila em Windows e em Linux a partir do **mesmo código-fonte**, sem alteração manual nenhuma. A seleção da API acontece em tempo de compilação, por diretivas `#ifdef _WIN32`.

3.1 Windows

```
.\scripts\compilar.ps1          # gera bin\servidor.exe e bin\cliente.exe
.\scripts\rodar_demo.ps1 -N 5   # sobe o servidor e dispara 5 clientes
```

No VS Code: Ctrl+Shift+B compila; a tarefa "Rodar demo (5 clientes)" executa a demonstração completa.

Para rodar manualmente, em duas janelas separadas:

```
# janela 1 -- servidor
.\bin\servidor.exe

# janela 2 -- cinco clientes ao mesmo tempo
1..5 | ForEach-Object { Start-Process -NoNewWindow .\bin\cliente.exe -ArgumentList $_ }
```

3.2 Linux / WSL

```
make                # compila para bin/
make servidor       # roda o servidor (em um terminal)
make demo N=5       # dispara 5 clientes  (em outro terminal)
make clean          # remove bin/
```

3.3 Argumentos do cliente

O cliente aceita três argumentos opcionais, úteis para testar entre máquinas diferentes:

```
cliente [rotulo] [host] [porta]

cliente 3                     # rotulo "3", conecta em 127.0.0.1:6090
cliente A 192.168.0.10 6090   # conecta em outra maquina da rede
```

Sem argumentos, assume rótulo "1" e o servidor em 127.0.0.1:6090.

4. Como funciona

4.1 Visão geral

O servidor é **concorrente**: o laço principal aceita conexões sem nunca bloquear e entrega cada uma a uma thread dedicada. Toda a serialização fica por conta do semáforo, dentro da thread.

```
thread principal                    thread de atendimento (uma por cliente)
----------------                    ---------------------------------------
accept()  ..... conexao 1 ------->  sem_descer()   <-- bloqueia aqui
accept()  ..... conexao 2 ------->  sem_descer()       se o recurso
accept()  ..... conexao 3 ------->  sem_descer()       estiver ocupado
accept()  ..... conexao 4 ------->  sem_descer()
   (nunca espera 2 s)                  |
                                       +--> le a hora, responde ao cliente
                                       +--> sleep 2 s  (recurso retido)
                                       +--> sem_subir()  libera o proximo
```

4.2 O semáforo

O semáforo é criado com valor inicial **1**: exatamente um atendimento por vez. A região crítica fica na função `atende_cliente()`, em `src/servidor.c`:

```
/* ====================== REGIAO CRITICA ====================== */
sem_descer(&recurso);           /* P()  -- so uma thread passa     */

    ... le o relogio local e responde ao cliente ...

/* O cliente ja foi atendido, mas o recurso segue retido por 2 s.
   So entao o proximo passa pelo sem_descer. */
sleep_ms(TEMPO_LOCK_MS);        /* 2000 ms                         */

sem_subir(&recurso);            /* V()  -- libera o proximo        */
/* ==================== FIM DA REGIAO CRITICA ================== */
```

A chamada de `sleep` fica **dentro** da região crítica, antes do `sem_subir()`. É isso que produz o espaçamento de 2 em 2 segundos: não é o atendimento que demora, é o recurso que permanece bloqueado.

4.3 Por que threads, e não um laço accept() sequencial?

Um servidor iterativo — como o do material da disciplina — também atenderia um cliente por vez. Mas essa serialização seria um *acidente da arquitetura*: não haveria semáforo nenhum, e o trabalho pede justamente um semáforo.

Com uma thread por conexão existe concorrência real: as cinco threads tentam entrar na região crítica ao mesmo tempo. A exclusão mútua passa a ser, de fato, responsabilidade do semáforo — que é o objeto de estudo. O material cobre esse modelo no slide *"Sockets Multithread"*.

4.4 Implementação do semáforo nas duas plataformas

Operação	Windows (Win32)	Linux (POSIX)
Criar (valor 1)	`CreateSemaphore`	`sem_init`
Descer — P() / wait	`WaitForSingleObject`	`sem_wait`
Subir — V() / post	`ReleaseSemaphore`	`sem_post`
Criar thread	`CreateThread`	`pthread_create`
Pausa de 2 s	`Sleep(2000)`	`usleep(2000000)`

Os dois conjuntos ficam atrás das mesmas três funções (`sem_criar`, `sem_descer`, `sem_subir`), de modo que o restante do código não sabe em qual sistema está rodando.

5. Protocolo de comunicação

Cliente e servidor trocam **8 bytes** em cada sentido: segundos e microssegundos desde 1º de janeiro de 1970, ambos em *network byte order* (big-endian), conforme a seção "Ordenação de Bytes" do material.

```
struct wire_time {
    uint32_t sec;    /* segundos desde 01/01/1970        */
    uint32_t usec;   /* microssegundos dentro do segundo */
};
```

O código original enviava a `struct timeval` diretamente, aplicando `htonl()` sobre `tv_sec`. Isso tem dois defeitos: `time_t` tem 64 bits nas plataformas atuais e `htonl()` converte apenas 32 deles, corrompendo o valor; e o tamanho de `struct timeval` difere entre Windows e Linux, o que impediria a comunicação entre sistemas diferentes.

A struct explícita de dois `uint32_t` resolve os dois problemas de uma vez — e isso foi comprovado na prática (seção 7.3).

5.1 Sequência de chamadas

```
SERVIDOR                              CLIENTE
--------                              -------
socket()                              socket()
setsockopt(SO_REUSEADDR)
bind()
listen()
accept()      <--- conexao TCP -----  connect()
                                      send()    envia a hora local
recv()        <--- 8 bytes ---------
  [ semaforo: P() ]
  le o relogio local
send()        ---- 8 bytes --------->  recv()    recebe a hora do servidor
  [ sleep 2 s; semaforo: V() ]
close()                               close()
```

6. Correções em relação ao código original

O código fornecido no Moodle, reproduzido nos slides da disciplina, não compilava no Windows e tinha defeitos que impediriam o trabalho de funcionar mesmo em Linux. Segue o que foi corrigido:

#	Problema no original	Correção
1	Código POSIX puro; não compila no Windows	camada `#ifdef _WIN32` com Winsock2
2	`for (j=0;j<1000000;j++);` não dá 2 s e é eliminado pelo otimizador do gcc	`Sleep(2000)` / `usleep(2000000)`
3	Servidor iterativo: sem concorrência e sem semáforo	uma thread por conexão + semáforo de contagem 1
4	`htonl()` sobre `time_t` de 64 bits truncava o valor	struct `wire_time` com dois `uint32_t`

#	Problema no original	Correção
5	`accept(..., (int *)0)` — tipo errado (`socklen_t *`)	`accept()` com `struct sockaddr_in` real
6	Sem SO_REUSEADDR: "Address already in use" ao reiniciar	`setsockopt(SO_REUSEADDR)`
7	`read`/`write` podiam transferir menos bytes que o pedido	laços `recebe_tudo` / `envia_tudo`
8	`bzero` e `<strings.h>` (obsoletos)	`memset` e `<string.h>`
9	`for (i=1; i<=50000; i++)` — limite arbitrário	laço infinito; encerra com Ctrl+C

O item 2 merece destaque numa apresentação: o laço vazio do código original é removido pelo compilador com `-O1` ou maior, porque não produz efeito observável. Mesmo sem otimização, um milhão de iterações vazias leva poucos milissegundos num processador atual — nunca 2 segundos.

7. Resultados dos testes

O projeto foi compilado e executado nos dois sistemas, sempre com `-O2 -Wall` e **sem erros nem avisos**.

	Windows	Linux
Compilador	gcc 16.1.0 (MinGW-W64 UCRT)	gcc 15.2.0 (Ubuntu 26.04 LTS, WSL2)
API de sockets	Winsock2 (-lws2_32)	BSD sockets
Threads	CreateThread	pthread_create (-pthread)
Semáforo	CreateSemaphore	sem_init / sem_wait / sem_post

7.1 Windows — 5 clientes simultâneos

```
[  3.363s] cliente 1 (127.0.0.1:55260) >>> RECURSO ADQUIRIDO
[  5.385s] cliente 2 (127.0.0.1:55261) >>> RECURSO ADQUIRIDO
[  7.393s] cliente 3 (127.0.0.1:55262) >>> RECURSO ADQUIRIDO
[  9.416s] cliente 4 (127.0.0.1:55263) >>> RECURSO ADQUIRIDO
[ 11.430s] cliente 5 (127.0.0.1:55264) >>> RECURSO ADQUIRIDO
```

Intervalos de 2,022 / 2,008 / 2,023 / 2,014 s. Total: 9,95 s.

7.2 Linux — 5 clientes simultâneos

```
[  1.044s] cliente 1 (127.0.0.1:48226) >>> RECURSO ADQUIRIDO
[  3.059s] cliente 2 (127.0.0.1:48230) >>> RECURSO ADQUIRIDO
[  5.060s] cliente 3 (127.0.0.1:48222) >>> RECURSO ADQUIRIDO
[  7.060s] cliente 4 (127.0.0.1:48242) >>> RECURSO ADQUIRIDO
[  9.061s] cliente 5 (127.0.0.1:48258) >>> RECURSO ADQUIRIDO

VALIDACAO
intervalo 1->2 : 2.015 s  [OK]
intervalo 2->3 : 2.001 s  [OK]
intervalo 3->4 : 2.000 s  [OK]
intervalo 4->5 : 2.001 s  [OK]
```

Total: 8,05 s.

Observe, nas duas saídas, que as cinco conexões chegam praticamente no mesmo instante — todas dentro de poucos milissegundos — e ainda assim os atendimentos saem separados por 2 segundos. É exatamente o que o enunciado pede.

7.3 Teste de interoperabilidade

Com o servidor rodando no **Linux** e quatro clientes no **Windows**, a serialização se manteve e as horas trafegaram corretamente nos dois sentidos. Isso comprova que a correção do protocolo (item 4 da seção 6) tornou o formato de fio compatível entre sistemas operacionais diferentes — algo que o código original não conseguiria.

Durante esse teste o servidor recebeu, por acaso, uma conexão que fechou sem enviar dados. Ele tratou o erro de leitura, liberou o semáforo e seguiu atendendo normalmente, sem travar nem perder o recurso — um caso de robustez que vale mencionar na apresentação.

8. Roteiro sugerido para a apresentação

Passo	O que fazer	O que destacar
1	Abrir `src/servidor.c` na função `atende_cliente()`	A região crítica: P(), atendimento, sleep de 2 s, V()
2	Rodar `.\scripts\rodar_demo.ps1 -N 5`	As 5 conexões chegam juntas; os atendimentos saem de 2 em 2 s
3	Mostrar a janela do servidor	Linhas "conectado" quase simultâneas vs. "RECURSO ADQUIRIDO" espaçadas
4	Mostrar a saída dos clientes	O campo "esperou N s" crescendo: 0, 2, 4, 6, 8
5	Comentar a tabela da seção 6	Por que o laço vazio do original não funcionaria

8.1 Perguntas prováveis

E se o cliente travar no meio do atendimento?

O servidor usa `recv` bloqueante, então a thread ficaria presa segurando o semáforo. Uma evolução natural seria aplicar `SO_RCVTIMEO` ao socket.

Por que o semáforo começa em 1, e não em N?

Valor 1 significa exclusão mútua — um atendimento por vez. Um valor N permitiria N atendimentos simultâneos, o que não é o que o enunciado pede.

Os clientes são atendidos na ordem em que chegaram?

Não necessariamente. O semáforo garante *exclusão mútua*, não ordem de chegada — quem acorda primeiro depende do escalonador do sistema operacional. Isso aparece nas saídas: os números dos clientes nem sempre saem em sequência.

Daria para usar processos em vez de threads?

Sim, com `fork()`, mas o semáforo teria de ficar em memória compartilhada (`sem_open` ou `shmget`). Threads compartilham o espaço de endereçamento de graça, e a aula anterior já levantava o custo dos "processos pesados".

9. Referências

- Material da disciplina — *Comunicação entre processos usando Sockets*: API socket, ordenação de bytes e a aplicação de tempo que serviu de base para este trabalho.
- Material da disciplina — *Sistemas Distribuídos*: definição, propriedades e transparência de concorrência.
- COMER, Douglas E. *Internetworking with TCP/IP*, vol. 1.
- TANENBAUM, Andrew. *Sistemas Distribuídos*.